Name - Chaitanya Kelkar

Roll no. - 53

Experiment 5

Aim - Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

2. Dataset Source & Description

- **Source:** Kaggle (Crop_recommendation.csv)
- **Description:** Multivariate agricultural data used to classify and recommend the optimal crop based on soil and weather attributes.
- **Size:** 2200 instances, 8 columns.
- **Target Variable:** label (22 crop categories).
- **Features:** N, P, K (Soil Nutrients) and temperature, humidity, ph, rainfall (Environmental Conditions).

3. Mathematical Formulation of the Algorithm

A Linear SVM finds the optimal straight-line boundary (hyperplane) that separates different classes while maximizing the "margin" between the closest data points (Support Vectors).

- **A. The Hyperplane Equation:** $w \cdot x + b = 0$
  (Where 'w' is the weight vector, 'x' is the input vector, and 'b' is the bias)
- **B. Margin Maximization:** To maximize the margin, the algorithm minimizes the magnitude of the weights:
  Minimize: $(1/2) ||w||^2$
- **C. Linear Kernel:** Unlike RBF, the linear kernel does not warp data into higher dimensions. It calculates the simple dot product:
  $K(x_i, x_j) = x_i \cdot x_j$

4. Algorithm Limitations

- **Linear Assumption:** It assumes the dataset can be separated by a straight line or flat plane. If the data is highly complex or circular, a linear SVM will underperform compared to an RBF kernel.
- **Feature Scaling Dependent:** SVM is highly sensitive to the magnitude of data points. If features are not scaled (e.g., Rainfall vs. pH), the hyperplane will be heavily skewed toward the larger numbers.

5. Methodology / Workflow

The experiment followed this classification pipeline:

- **Data Preprocessing:** Handled features (X) and target (y), followed by a Stratified split (80% Train, 20% Test).
- **Standardization:** Applied StandardScaler to ensure all features contributed equally to the margin distance.
- **Model Tuning:** Used GridSearchCV restricted specifically to `kernel='linear'` to find the best configuration.
- **Feature Importance Extraction:** Because the model is linear, the weights (`.coef_`) were extracted to rank which environmental factors drove the model's decisions.
- **Dimensionality Reduction:** Used Principal Component Analysis (PCA) to compress 7 features into 2 so the hyperplanes could be graphed.

6. Performance Analysis

- **Confusion Matrix:** Showed high precision across the 22 crops, accurately mapping True Positives and False Positives.
- **Feature Importance:** The linear weights successfully proved exactly which factors (like Rainfall or Humidity) were most critical for predicting specific crop types.
- **Hyperplane Visualization:** The PCA projection successfully mapped the multidimensional data onto a 2D plane, visually proving how the SVM drew straight-line boundaries to separate different crop clusters.

7. Hyperparameter Tuning

GridSearchCV mathematically located the best possible value for the following parameter:

- **C (Regularization Parameter):** Controls the penalty for misclassifying data points.
    - *Low C:* Creates a "Soft Margin" that allows some misclassifications but creates a wider, more generalized boundary.
    - *High C:* Creates a "Hard Margin" that strictly penalizes errors, drawing a very tight boundary (which risks overfitting to the training data).

Code -

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.decomposition import PCA
```

```python
# 1. DATA LOADING & EXPLORATION
print("Loading Data...")
try:
    df = pd.read_csv('../data/Crop_recommendation.csv')
    print("Data loaded successfully.")
except FileNotFoundError:
    print("Error: File not found. Check your 'data' folder structure.")
    exit()

print("\n--- Data Exploration ---")
print(df.head())
print('\nMissing values:\n', df.isnull().sum())

# 2. DATA PREPROCESSING & SPLITTING
X = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]
y = df['label']

# Stratify=y ensures all 22 crops are distributed evenly
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

# Feature Scaling (Crucial for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. LINEAR SVM & HYPERPARAMETER TUNING
print("\n--- Starting Linear SVM Hyperparameter Tuning ---")
print("Testing combinations of C (Regularization)...")

# We strictly set kernel to 'linear' so we can extract Feature Importance
later
param_grid = {
    'C': [0.1, 1, 10, 100],
    'kernel': ['linear']
}

svm_base = SVC(random_state=42)
```

```
grid_search = GridSearchCV(estimator=svm_base, param_grid=param_grid,
cv=5, verbose=1, n_jobs=-1)

grid_search.fit(X_train_scaled, y_train)
best_svm = grid_search.best_estimator_

print("BEST HYPERPARAMETERS FOUND:")
print(f"Kernel Type: {grid_search.best_params_['kernel']}")
print(f"C (Regularization): {grid_search.best_params_['C']}")

# 4. EVALUATION & CONFUSION MATRIX
y_pred = best_svm.predict(X_test_scaled)

print("--- Classification Report ---")
acc = accuracy_score(y_test, y_pred)
print(f"Linear SVM Accuracy: {acc*100:.2f}%\n")
print(classification_report(y_test, y_pred))

print("\nGenerating Confusion Matrix...")
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(12, 10))
sns.heatmap(cm, annot=True, fmt='d', cmap='rocket',
            xticklabels=best_svm.classes_,
            yticklabels=best_svm.classes_)

plt.title(f"Linear SVM Confusion Matrix (C:
{grid_search.best_params_['C']})", fontsize=16)
plt.xlabel("Predicted Label", fontsize=12)
plt.ylabel("True Label", fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig("exp5_linear_svm_confusion_matrix.png")
print("Saved 'exp5_linear_svm_confusion_matrix.png'")

# 5. FEATURE IMPORTANCE
print("\nGenerating Feature Importance Chart...")

# Average the absolute weights across all 22 crop classes
coef = np.abs(best_svm.coef_).mean(axis=0)
```

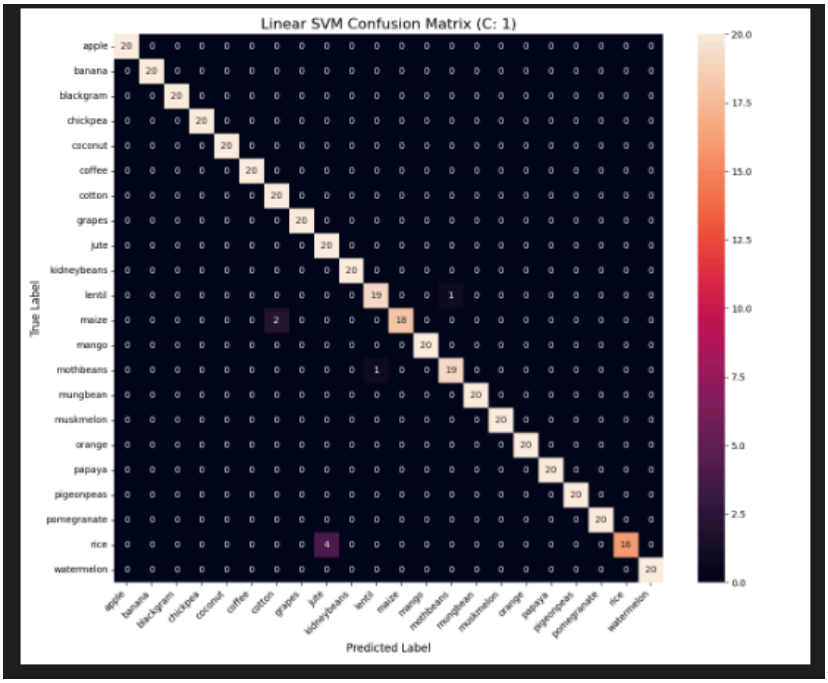
```
features = X.columns

imp_df = pd.DataFrame({'Feature': features, 'Importance': coef})
imp_df = imp_df.sort_values('Importance', ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=imp_df, palette='viridis')
plt.title('Feature Importance (Linear SVM)')
plt.tight_layout()
plt.savefig('exp5_linear_svm_feature_importance.png')
print("Saved 'exp5_linear_svm_feature_importance.png'")


# 6. 2D HYPERPLANE VISUALIZATION (PCA)
print("\nGenerating 2D Hyperplane Visualization using PCA...")

# Compress the 7 features into 2 so we can plot them on an X/Y axis
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)

# Convert string labels into numbers for plotting colors
le = LabelEncoder()
y_train_num = le.fit_transform(y_train)

# Train a 2D Linear SVM strictly for the graph
svm_2d = SVC(kernel='linear', C=grid_search.best_params_['C'])
svm_2d.fit(X_train_pca, y_train_num)

# Create a mesh grid to plot the hyperplanes (decision boundaries)
x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                     np.arange(y_min, y_max, 0.1))

Z = svm_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(10, 8))
plt.contourf(xx, yy, Z, alpha=0.4, cmap='tab20')
plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train_num,
```

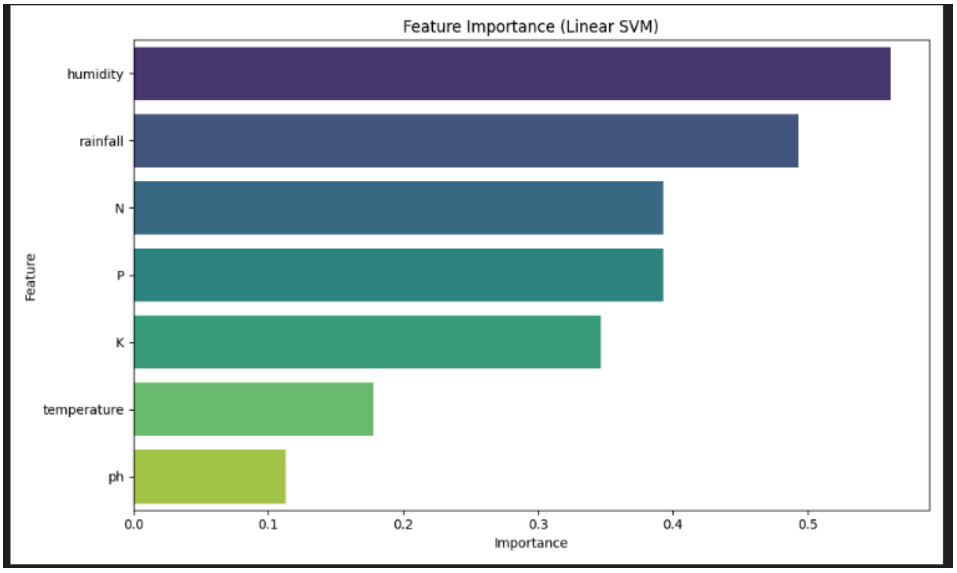
```
            edgecolor='k', cmap='tab20', s=20)

plt.title(f"Linear SVM Hyperplanes (2D PCA Projection)\nC:
{grid_search.best_params_['C']}", fontsize=14)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.tight_layout()
plt.savefig("exp5_linear_svm_hyperplanes_2d.png")
print("Saved 'exp5_linear_svm_hyperplanes_2d.png'. Experiment Complete!")
```

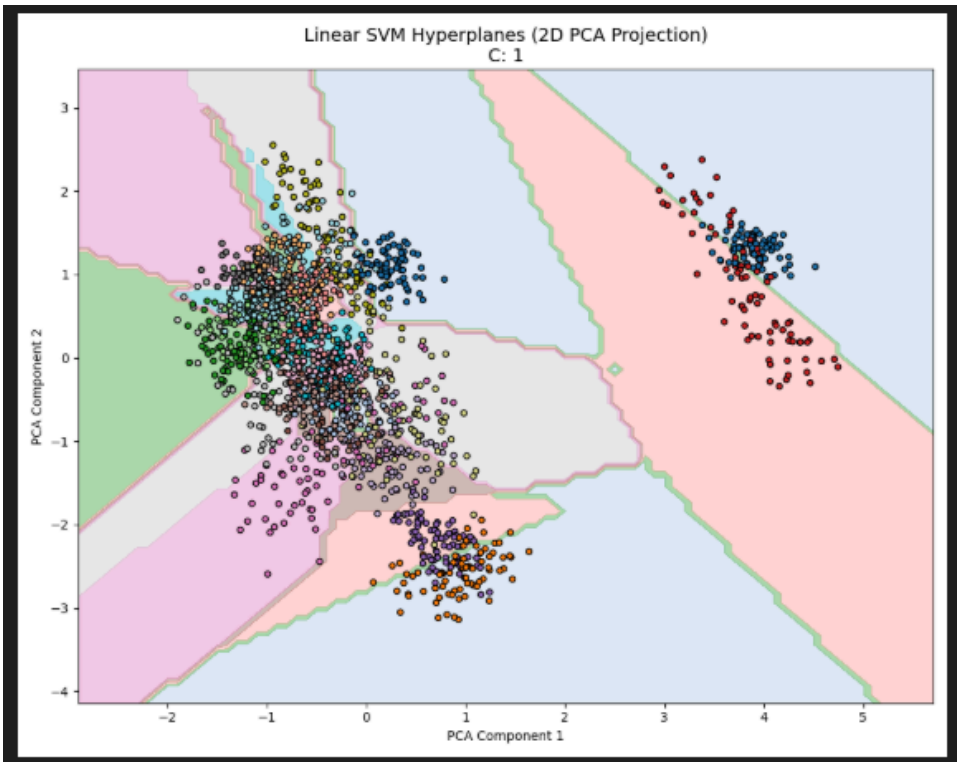
Output -

```
PS D:\SmartCropProject\src> python svm.py
Loading Data...
Data loaded successfully.

--- Data Exploration ---
    N   P   K  temperature   humidity        ph    rainfall label
0  90  42  43    20.879744  82.002744  6.502985  202.935536  rice
1  85  58  41    21.770462  80.319644  7.038096  226.655537  rice
2  60  55  44    23.004459  82.320763  7.840207  263.964248  rice
3  74  35  40    26.491096  80.158363  6.980401  242.864034  rice
4  78  42  42    20.130175  81.604873  7.628473  262.717340  rice

Missing values:
 N              0
P               0
K               0
temperature     0
humidity        0
ph              0
rainfall        0
label           0
dtype: int64

--- Starting Linear SVM Hyperparameter Tuning ---
Testing combinations of C (Regularization)...
Fitting 5 folds for each of 4 candidates, totalling 20 fits
BEST HYPERPARAMETERS FOUND:
Kernel Type: linear
C (Regularization): 1
--- Classification Report ---
Linear SVM Accuracy: 98.18%
```

```
              precision    recall  f1-score   support

       apple       1.00      1.00      1.00        20
      banana       1.00      1.00      1.00        20
   blackgram       1.00      1.00      1.00        20
    chickpea       1.00      1.00      1.00        20
     coconut       1.00      1.00      1.00        20
      coffee       1.00      1.00      1.00        20
      cotton       0.91      1.00      0.95        20
      grapes       1.00      1.00      1.00        20
        jute       0.83      1.00      0.91        20
 kidneybeans       1.00      1.00      1.00        20
      lentil       0.95      0.95      0.95        20
       maize       1.00      0.90      0.95        20
       mango       1.00      1.00      1.00        20
   mothbeans       0.95      0.95      0.95        20
    mungbean       1.00      1.00      1.00        20
   muskmelon       1.00      1.00      1.00        20
      orange       1.00      1.00      1.00        20
      papaya       1.00      1.00      1.00        20
  pigeonpeas       1.00      1.00      1.00        20
 pomegranate       1.00      1.00      1.00        20
        rice       1.00      0.80      0.89        20
  watermelon       1.00      1.00      1.00        20

    accuracy                           0.98       440
   macro avg       0.98      0.98      0.98       440
weighted avg       0.98      0.98      0.98       440


Generating Confusion Matrix...
Saved 'exp5_linear_svm_confusion_matrix.png'

Generating Feature Importance Chart...
```




Feature Importance (Linear SVM)
humidity
rainfall
N
P
K
temperature
ph
Feature
0.0
0.1
0.2
0.3
0.4
0.5
Importance


Linear SVM Hyperplanes (2D PCA Projection)
C: 1
PCA Component 2
PCA Component 1
3
2
1
0
−1
−2
−3
−4
−2
−1
0
1
2
3
4
5